

# Java Conditional Statements

## Java Conditional Statements

1. Flow of Control
2. The `if` Statement
3. The `if-else` Statement
4. Nested `if` Statements
5. The `if-else-if` Ladder
6. The `switch` Statement
7. Switch vs. `if-else-if` Ladder
8. Nested `switch` Statements

★ One-Page Revision

📝 Practice Questions

🔥 Quick FAQ

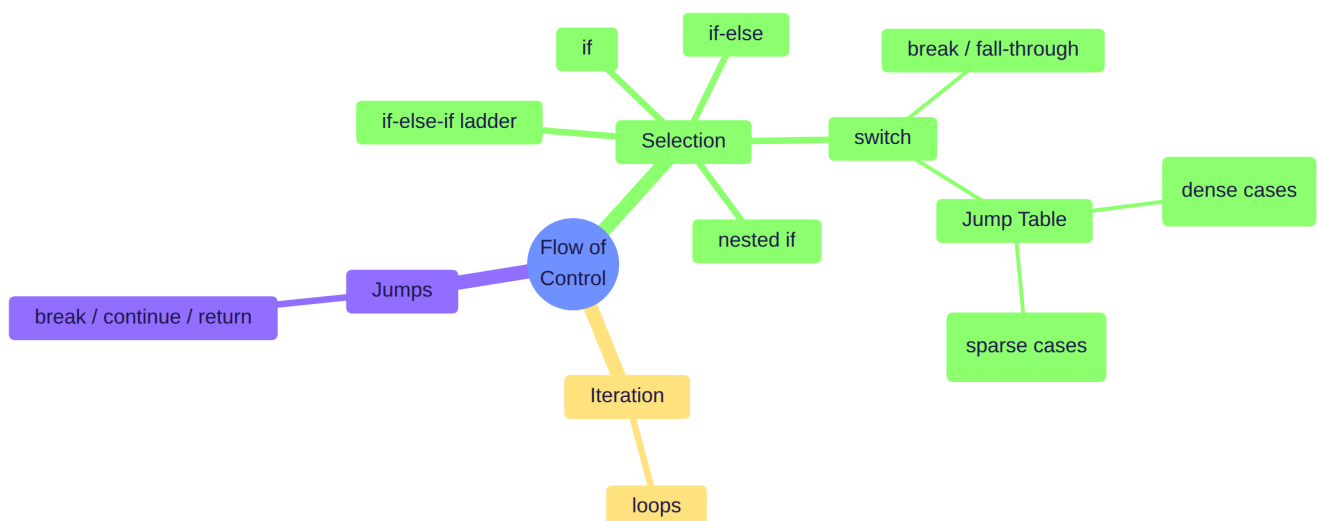
# Java Conditional Statements

`if`, `if-else`, Nested `if`, `if-else-if` Ladder, `Switch` — and Why `Switch` Is Faster

## 1. Flow of Control

**Definition:** Flow of control is the order in which statements in a program actually execute.

By default, Java code executes **sequentially** — line 1, then line 2, then line 3, and so on. Certain keywords break this straight-line execution and redirect the program down a different path. Java groups these into three categories:



| Category  | Purpose                                                                                                             |
|-----------|---------------------------------------------------------------------------------------------------------------------|
| Selection | Choose between different paths ( <code>if</code> , <code>if-else</code> , <code>switch</code> )                     |
| Iteration | Repeat a block of code (loops)                                                                                      |
| Jumps     | Immediately exit or skip parts of a code block ( <code>break</code> , <code>continue</code> , <code>return</code> ) |

This lecture covers **Selection** in full.

## 2. The `if` Statement

**Definition:** `if` evaluates an **expression** — which must resolve to `true` or `false` — and executes its code block **only** if that expression is `true`. If `false`, the block is skipped entirely.

**Syntax:**

```
if (expression) {
    // do something
}
```

**Example:**

```
boolean b = true;
if (b == true) {
    System.out.println("b is true");
}
```

### 💡 Easy Hinglish Explanation

"`if` ka matlab hai: agar ye expression true hai, tabhi is block ke andar ka code chalega. False hai to seedha skip ho jaayega, jo neeche likha hai wahi chalega."

📌 **Important:** The expression inside `if(...)` isn't limited to simple `true` / `false` variables — it can be any comparison or logical expression that evaluates to a boolean, such as `i == 5` or `i > 5 && i < 10`.

## 3. The `if-else` Statement

A plain `if` only decides whether to run a block or skip it. Adding `else` creates a true **two-way fork** — exactly one of the two blocks always runs.

**Syntax:**

```
if (expression) {
    // runs if expression is true
} else {
    // runs if expression is false
}
```

**Example:**

```
int i = 5;
if (i == 5) {
    System.out.println("i is five");
} else {
    System.out.println("i is not five");
}
```

⚠ **Common Mistake:** Writing two separate `if` blocks instead of `if-else` when only one outcome should happen. With two independent `if` blocks, **both** could technically execute if you're not careful with the logic — `if-else` guarantees mutual exclusivity.

## Curly Braces Are Optional for Single Statements

If the `if` or `else` block contains **only one statement**, the curly braces `{ }` become optional:

```
if (i == 5)
    System.out.println("i is five");
else
    System.out.println("i is not five");
```

✅ **Best Practice:** Even though braces are optional for single statements, **always include them anyway**. If you later add a second line to the block, forgetting to add braces at that point is a classic source of bugs.

## 4. Nested `if` Statements

**Definition:** A "nested if" is simply an `if` statement placed **inside** another `if` (or `else`) block, allowing multi-layered decisions.

**Example:**

```
if (i > 5) {
    if (i < 10) {
        // runs only if i is BOTH > 5 AND < 10
    } else {
        // runs if i > 5 but NOT < 10
    }
} else {
    // runs if i is not > 5 at all
}
```

**Rule for matching `else` to `if`:** Each `else` always belongs to the **nearest preceding `if`** at the same nesting level. In the example above, the inner `else` belongs to the inner `if (i < 10)`, and the outer `else` belongs to the outer `if (i > 5)`.

🔴 **Important:** Not every `if` needs an `else`. The innermost `if` in a nested chain can be left without one — if its condition is `false`, execution simply continues past it with no action taken.

## Prefer Combined Conditions Over Nesting

The two-level nested example above (`i > 5` then `i < 10`) can be written more simply using a **logical AND**:

```
if (i > 5 && i < 10) {
    // same result, one condition instead of two nested ifs
}
```

✓ **Best Practice:** Java allows nesting to **any depth**, but good coding practice limits it to a **maximum of 2 levels**. Beyond that, code becomes hard to trace ("if this, then if that, then if this other thing..."), so combining conditions with logical operators ( `&&` , `||` ) is almost always the cleaner choice.

## 5. The `if-else-if` Ladder

**Definition:** A chain of `if / else if / else if / ... / else` used when there are **multiple, mutually exclusive** choices to test — not just two.

**Syntax:**

```
if (i == 5) {
    // ...
} else if (i == 6) {
    // ...
} else if (i == 7) {
    // ...
} else {
    // runs only if none of the above matched
}
```

**How it executes:** Java checks each condition **top to bottom**. The moment one evaluates `true`, that block runs, and **every remaining condition in the ladder is skipped entirely** — control jumps straight out of the whole ladder.

**Worked Example — Age Categories:**

```
int age = 50;

if (age > 80) {
    System.out.println("You are very old");
} else if (age > 60) {
    System.out.println("You are old");
} else if (age > 40) {
    System.out.println("You are becoming old");
} else if (age > 20) {
    System.out.println("You are young");
}
```

For `age = 50`, the first two conditions ( `> 80` , `> 60` ) are `false`, but `age > 40` is `true` — so `"You are becoming old"` prints, and the remaining `else if` is never even evaluated.

★ **Exam Tip:** In an `if-else-if` ladder, **only one block ever executes** — even if multiple conditions would technically be `true`, only the *first* matching one runs.

## 6. The `switch` Statement

**Definition:** `switch` is an alternative to a large `if-else-if` ladder, designed specifically for testing **equality** against a set of fixed, unique values.

### Syntax:

```
switch (i) {
    case 1:
        System.out.println("i is one");
        break;
    case 2:
        System.out.println("i is two");
        break;
    case 3:
        System.out.println("i is three");
        break;
    default:
        System.out.println("i is greater than 3");
        break;
}
```

### How it maps to if-else-if:

| switch                 | Equivalent if-else-if         |
|------------------------|-------------------------------|
| <code>switch(i)</code> | —                             |
| <code>case 1:</code>   | <code>if (i == 1)</code>      |
| <code>case 2:</code>   | <code>else if (i == 2)</code> |
| <code>default:</code>  | <code>else</code>             |

### Rules for `switch`

🔴 **Important** — **allowed types:** The expression inside `switch(...)` must evaluate to one of: `byte`, `short`, `int`, `char` (or, as covered below, `String` from JDK 7 onward, and `enum` types). **`boolean` is not allowed** — attempting `switch(i > 4)` gives a compile error: *"cannot switch on a value of type boolean."*

🔴 **Important** — **unique cases:** Every `case` value must be **unique**. Writing two `case 1:` labels in the same switch causes a **duplicate case** compile error — unlike an `if-else-if` ladder, where writing the same condition twice is technically legal (the second one is just never reached).

### The `break` Keyword and Fall-Through

⚠️ **Common Mistake:** Forgetting `break` at the end of a case doesn't stop execution — it causes **fall-through**: control continues into the *next* case's code, regardless of whether that case's value actually matches, all the way until it hits a `break` (or the end of the switch).

**Example without any `break` statements ( `i = 1` ):**

```

switch (i) {
    case 1:
        System.out.println("i is 1");
    case 2:
        System.out.println("i is 2");
    case 3:
        System.out.println("i is 3");
    default:
        System.out.println("i is greater than 3");
}

```

**Output:**

```

i is 1
i is 2
i is 3
i is greater than 3

```

Even though `i` is `1`, **every** case printed — because without `break`, execution just keeps falling into the next block below it.

 **Easy Hinglish Explanation**

"`break` na likho to jo bhi match hua uske baad wale saare cases bhi chalte chale jaayenge — chahe unka condition match ho ya na ho. `break` likhne se hum ek particular case ke andar hi rukte hain."

✓ **Best Practice:** Fall-through isn't a bug in the language — it's a deliberate feature Java gives you, for situations where you genuinely want multiple cases to execute together once a match is found. But unless you specifically intend that, **always include `break`** at the end of every case, including `default`.

**From JDK 7 Onward: String Switches**

Before JDK 7, `switch` only worked on `byte` / `short` / `int` / `char` (and `enum`). **From JDK 7 onward, `String` can also be used as a switch expression.**

🔥 **Important:** After JDK 14, `switch` received major syntax enhancements (not covered here, since they build on concepts like arrays and expressions not yet introduced). The traditional `switch` syntax shown above remains fully valid and supported today — the JDK 14+ changes only *add* new capabilities, they don't remove the old ones.

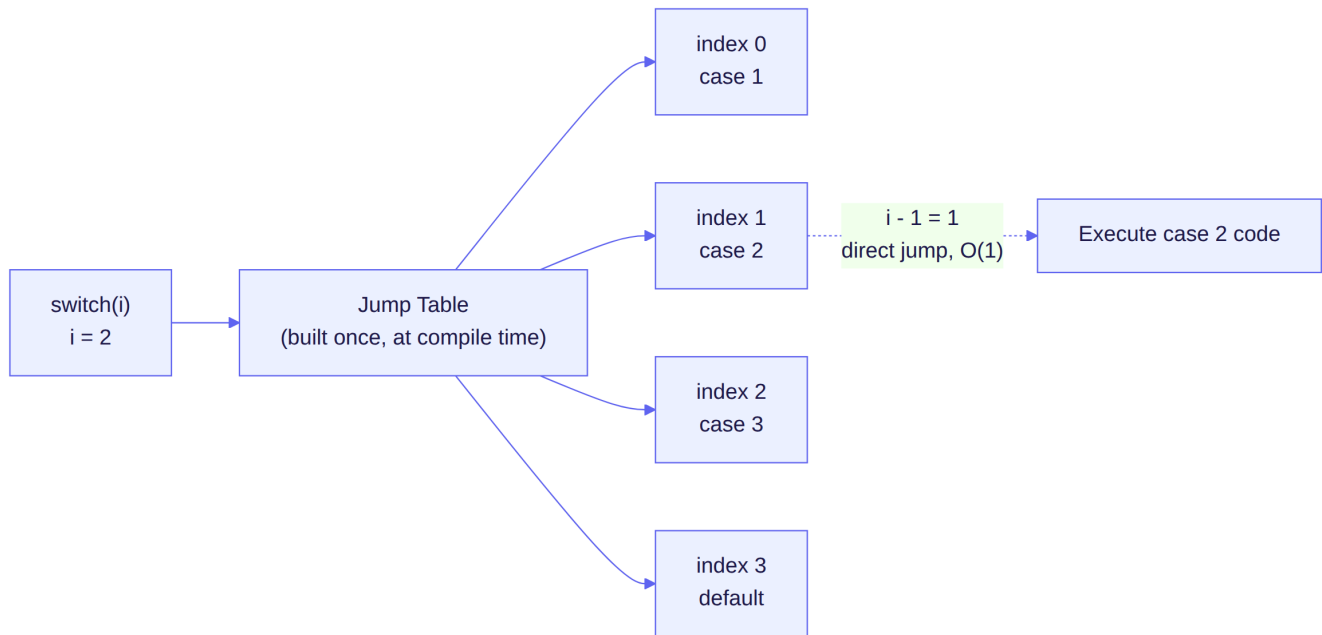
**7. Switch vs. if-else-if Ladder**

Both can look interchangeable for simple cases, but they differ in two important ways:

|                                  | <b>if-else-if</b> Ladder                                                                                                                            | <b>switch</b>                                                      |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| <b>Can test</b>                  | Equality <i>and</i> inequality ( <code>==</code> , <code>&gt;</code> , <code>&lt;</code> , <code>&amp;&amp;</code> , etc. — any boolean expression) | <b>Only equality</b> against fixed constant values                 |
| <b>Speed for large case sets</b> | $O(n)$ — checks conditions one by one                                                                                                               | $O(1)$ or $O(\log n)$ — jumps almost directly to the matching case |
| <b>Duplicate values</b>          | Allowed (just unreachable)                                                                                                                          | <b>Not allowed</b> — compile error                                 |

## Why Is `switch` Faster? — Jump Tables

When the JVM compiles a `switch` statement, it doesn't check `case 1`, then `case 2`, then `case 3` one at a time like an `if-else-if` ladder does. Instead, it pre-builds an internal lookup structure called a **jump table**, so it can go **directly** to the matching case.



**Definition:** A jump table works conceptually like an **array**. Each `case` value's code is placed at a computed index (roughly `case value - 1`), so when `i` comes in, the JVM computes the index directly and jumps straight there — no comparisons needed.

Concrete comparison for `i = 3`:

- **if-else-if ladder:** Checks `i == 1` (false) → checks `i == 2` (false) → checks `i == 3` (true) → executes → **3 evaluations total**
- **switch with a jump table:** Computes the index directly from `i`, jumps straight to `case 3` → **1 evaluation total**

★ **Exam Tip — Time Complexity**

`if-else-if` ladder = **O(n)** (linear scan through conditions) `switch` with a jump table = **O(1)** (direct jump)

## Jump Tables Aren't Always Built — Table Switch vs. Lookup Switch

A true array-style jump table only makes sense when the case values are **densely packed** (e.g. `1, 2, 3, 4`). If the compiler tried to build a direct-index array for widely spread-out values like `case 1`, `case 1000`, `case 100000`, it would need an enormous array with almost every slot sitting empty — extremely wasteful.

So the compiler picks **one of two strategies**, based on how the case values are distributed:

| Strategy             | Used When                                                                     | Mechanism                                                               | Time Complexity |
|----------------------|-------------------------------------------------------------------------------|-------------------------------------------------------------------------|-----------------|
| <b>Table Switch</b>  | Case values are <b>dense</b> (close together, e.g. <code>1, 2, 3, 4</code> )  | Direct array-style indexing                                             | <b>O(1)</b>     |
| <b>Lookup Switch</b> | Case values are <b>sparse</b> (far apart, e.g. <code>1, 1000, 100000</code> ) | Binary search over a small, sorted table of just the actual case values | <b>O(log n)</b> |

## 💡 Easy Hinglish Explanation

"Agar cases paas-paas hain (1,2,3,4) to compiler ek seedha table bana deta hai — direct jump ho jaata hai,  $O(1)$ . Lekin agar cases bahut door-door hain (1, 1000, 1 lakh), to itna bada table banana waste hoga. Isliye compiler sirf jitne cases hain utna hi chhota table banata hai, aur usme binary search lagata hai —  $O(\log n)$ ."

🔴 **Important:** Even the **slower** of the two options (Lookup Switch, at  $O(\log n)$ ) is still faster than a plain **if-else-if** ladder's  $O(n)$  for a large number of cases. This is the real reason **switch** is described as "more optimized."

★ **Exam Tip — why **if-else-if** still exists:** Since **switch** is limited to equality checks on **byte** / **short** / **int** / **char** / **String** / **enum**, it **cannot** replace **if-else-if** for range checks (  **$i > 5$**  ), complex boolean logic (  **$i > 5 \ \&\& \ i < 10$**  ), or comparisons on unsupported types like **boolean** or **double**. Each construct has a distinct, non-overlapping use case.

## 8. Nested **switch** Statements

Just like **if** can be nested inside **if**, a **switch** can be nested inside another **switch** — including inside a specific **case** block.

**Example:**

```
int j = 5;
switch (j) {
    case 4:
        System.out.println("j is 4");
        break;
    case 5:
        int k = 1;
        switch (k) {
            case 1:
                System.out.println("k is 1");
                break;
            case 2:
                System.out.println("k is 2");
                break;
        }
        break;
    default:
        System.out.println("j is not 4 and 5");
        break;
}
```

There's no fixed limit on nesting depth — you could nest a **switch** inside a **switch** inside a **switch**. But exactly like nested **if** s:

✅ **Best Practice:** Only nest a **switch** when your actual logic genuinely requires checking a second, independent value after matching the first. Avoid nesting purely out of habit — it adds complexity for the reader with no benefit if a flatter structure would work just as well.

## ★ One-Page Revision

### Core Definitions

- **Flow of control** = the order code actually executes in; normally sequential, altered by Selection / Iteration / Jumps



- **if** = runs a block only if its boolean expression is **true**
- **if-else** = guarantees exactly one of two blocks runs
- **Nested if** = an **if** inside another **if**; each **else** binds to its nearest enclosing **if**
- **if-else-if ladder** = multiple mutually exclusive conditions; only the first **true** one executes
- **switch** = tests **equality only**, against **byte** / **short** / **int** / **char** / **String** (JDK 7+)/ **enum**
- **Fall-through** = without **break**, execution continues into the next case regardless of its condition
- **Jump table** = an internal array-like structure the JVM builds so **switch** can jump directly to the matching case

### Key Rules

- **switch** cannot use **boolean** or evaluate inequalities (**>**, **<**) — use **if-else-if** for those
- **case** values must be unique; duplicates cause a compile error
- Always include **break** unless intentional fall-through is the goal
- Curly braces are optional for single-statement blocks, but including them is best practice
- Max recommended nesting depth: **2 levels** — prefer **&&** / **||** to combine conditions instead

**Complexity Comparison** | Construct | Time Complexity | ---|---| | **if-else-if** ladder | O(n) | | **switch** — Table Switch (dense cases) | O(1) | | **switch** — Lookup Switch (sparse cases) | O(log n) |



## Practice Questions

### Basic

1. What is the difference between **if** and **if-else**?
2. Why can't you write **switch(i > 4)** in Java?
3. What does **break** do inside a **switch** case?

**Medium** 4. Given **int i = 6;** and an **if-else-if** ladder checking **i == 5**, **i == 6**, **i == 7**, which block executes, and how many conditions get evaluated? 5. Rewrite this nested **if** using a single combined condition:

```
if (i > 5) {
    if (i < 10) {
        // do something
    }
}
```

6. What happens if you remove **break** from every case in a switch, and **i** matches the first case?

**Interview** 7. Why is **switch** generally faster than an **if-else-if** ladder for a large number of cases? 8. What's the difference between a Table Switch and a Lookup Switch, and what decides which one the compiler builds? 9. Since **switch** is more optimized, why does **if-else-if** still exist as a language feature? 10. Can two **case** labels in the same **switch** have the same value? What about two identical conditions in an **if-else-if** ladder?



## Quick FAQ

Can **switch** test ranges, like **case > 5**? No — **switch** only tests exact equality against constant values. Use **if-else-if** for ranges.

Is fall-through always a bug? No — it's occasionally used intentionally, when multiple case values should trigger the same combined behavior. But unintentional fall-through (forgetting **break**) is a common source of bugs.

**Does `switch` work with `double` or `boolean` ?** No. Only `byte` , `short` , `int` , `char` , `String` (JDK 7+), and `enum` types are allowed.